

T.C.AMASYA ÜNİVERSİTESİ
TEKNOLOJİ FAKÜLTESİ

BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

BİTİRME ÇALIŞMASI

FİNANSAL UZLAŞMA BACKEND PROJESİ

201902050 - Rümeysa FİLİZ

Danışman
Bekir PARLAK

Haziran 2024

AMASYA

T.C. AMASYA ÜNİVERSİTESİ

TEKNOLOJİ FAKÜLTESİ

Haziran - 2024

FİNANSAL UZLAŞMA WEB PROJESİ

Finansal Uzlaşma Backend Projesi 201902050- Rûmeysa FİLİZ

ÖZET

Bu bitirme çalışmasının amacı, şirketler arası finansal uzlaşma süreçlerini otomatikleştiren ve şirketler arası hesap anlaşmalarını kolaylaştıran bir web tabanlı backend geliştirmek. Bitirme çalışmamı .Net ile Katmanlı Mimari (Monolith Architecture) kullanarak gerçekleştirdim. Bitirme çalışmamda veri tabanı olarak SQL (Structured Query Language) kullandım. Genel olarak backend bir ORM (Object Relational Mapping) yapısı sahiptir. ORM yapısını kurabilmek için ise Entity Framework kütüphanesinden destek aldım. Entity Framework aynı zamanda veri tabanı ile backend servislerimin arasında iletişim sağlamaktadır

Anahtar Kelimeler : .NET, katmanlı mimari, entity framework, backend, ORM, SQL, C#, yazılım, finansal uzlaşma, web tabanlı backend

TEŞEKKÜR

Çalışmalarım boyunca değerli yardım ve katkılarıyla beni yönlendiren Hocam Doç. Dr. Bekir PARLAK'A ayrıca manevi destekleriyle beni hiçbir zaman yalnız bırakmayan çok değerli arkadaşlarım Fulden ÖZSAYIN ve Nurcan KIZKABAN'a teşekkürü bir borç bilirim.

KISALTMALAR

Bu çalışmada kullanılmış kısaltmalar, açıklamaları ile birlikte aşağıda sunulmuştur.

Kısaltmalar

Açıklama

DI

Dependency Injection

EF

Entity Framework

JSON

JavaScript Object Notation

MSSQL

Microsoft Structured Query Language

ORM

Object Relational Mapping

İÇİNDEKİLER

	Sayfa
ÖZET	i
TEŞEKKÜR.....	ii
İÇİNDEKİLER	iii
KISALTMALAR	iv
1. GİRİŞ	1-2
2. WEB TABANLI UYGULAMALAR VE SİTELER	2-6
2.1. Web Nedir?	2-3
2.2. Web Gelişimi	3-4
2.3 Neden Web Tabanlı?	4-5
2.4 Web Sitesi	5-6
3. FİNANSAL UZLAŞMA.....	6-9
3.1. Fİnalsal Uzlaşma Nedir?	6
3.2. Nasıl Finansal Uzlaşma Yapılır?.....	6-7
3.3 Finansal Hesap Uzlaşması Nedir?	7
3.4 BA/BS Uzlaşmaı Nedir?	7-8
3.5 Finansal Uzlaşma Yöntemini Kimler Kullanabilir?	8
3.6 Finansal Uzlaşma Yararları Nelerdir?	8-9
3.7 Bu Projenin Etkisi.....	9
4. LİTERATÜR TARAMASI	9-13
4.1 Finansal uzlaşma Sislemleri : Tarihçe ve Gelişimi.....	9
4.2 Mevcut Çalışma ve Yöntemler.....	10
4.3 Teorik Çerçeve ve Temel Konular.....	10
4.3.1 Katmanlı Mimari.....	10-11
4.3.2 Veri Tabanı Yöntemi Sistemleri (VTYS)	11

4.3.3 Kullanıcı Yetkilendirme ve E-Mail Parametreleri.....	11-13
5. SİSTEM TASARIMI VE MİMARİSİ	13-20
5.1 Uygulama Katmanı (BackEnd)	13-14
5.2 Veri Tabanı Katmanı.....	14
5.3 Katmanlı Mimari.....	15
5.3.1 Neden Katmanlı Mimariye İhtiyaç Duyuyoruz?	15
5.3.2 Entities Katmanı.....	16
5.3.3 Core Katmanı.....	16
5.3.4 Business Katmanı.....	17
5.3.5 Data Acces Katmanı.....	17-18
5.3.6 Presentation Layer (Sunum Katmanı).....	18- 19
5.3.7 Wep Api.....	19
5.4 Veri Tabanı Tasarımı.....	19-20
6. TEKNOLOJİ VE ARAÇLAR	21-27
6.1 Net Core ve C#.....	21
6.2 Entitiy Framework.....	21-22
6.2.1 ORM nedir?	21
6.2.2 Neden Kullanılır?	21-22
6.3 SQL Server.....	22
6.4 API Geliştirme ve Entegrasyonu.....	22-23
6.5 Veri Yönetimi ve Güvenliği.....	23
6.5.1 Veri Doğrulama.....	23
6.5.2 Güvenlil Protokolleri.....	24
6.5.3 Erişim Kontrolü ve İzinleri.....	24
6.6 RESTful API İlkeleri.....	24-25
6.6.1 Rest Sırasında Kullanılan İstekler Nelerdir?	25
6.7 Swagger ve Api Dokümantasyonu.....	26-27

6.8 Postman.....	27
7. UYGULAMA GELİŞTİRME	27-35
7.1 CRUD.....	28-29
7.1.1 Create (Oluşturma)	28
7.1.2 Read (Okuma)	28
7.1.3 Update (Güncelleme)	28
7.1.4 Delete (Silme)	29
7.2 Generic Yapı.....	29-30
7.3 Dependency Injection.....	30-31
7.4 İşlem Sonuç Sistemi.....	31-32
7.5 FluentValidation.....	32
7.6 Hashing.....	32-33
7.7 Caching	34
7.8 Interceptor.....	35
8. TEST VE DEĞERLENDİRME	36-37
8.1 Birim Testleri.....	36
8.2 Entegrasyon Testleri.....	36
8.3 Performans Testleri.....	36
8.4 Test Sonuçları ve Değerlendirme.....	36-37
9. SONUÇ VE ÖNERİLER	37-38
9.1 Projenin Katkıları ve Sınırlamaları.....	37-38
9.2 Gelecek Çalışma ve Öneriler.....	38
KAYNAKLAR	39
ÖZGEÇMİŞ	40

1. GİRİŞ

Bu bitirme çalışmasının amacı, finansal uzlaşma süreçlerini otomatikleştiren ve şirketler arası hesap uzlaşmasını kolaylaştıran bir web tabanlı backend geliştirmektir. Finansal uzlaşma sistemleri, finansal verilerin karşılaştırılması ve doğrulanması sürecinde manuel işlemleri en aza indirerek, hata oranlarını düşürmekte ve süreç verimliliğini artırmaktadır. Bu çalışmada geliştirilecek sistem, kullanıcı dostu arayüzü ve güçlü altyapısı ile işletmelerin uzlaşma süreçlerini daha hızlı ve güvenli bir şekilde yönetmelerine olanak tanıyacaktır.

Finansal uzlaşma sistemleri, özellikle büyük ölçekli işletmelerde ve yoğun işlem hacmine sahip firmalarda önemli bir ihtiyaç haline gelmiştir. Bu tür sistemler, manuel işlemlerden kaynaklanan hataların önüne geçerken, aynı zamanda iş süreçlerinin hızlanmasını sağlamaktadır. Çalışmanın temel hedefleri şu şekildedir:

- Kullanıcı dostu ve erişilebilir bir arayüz tasarlamak.
- Güvenli veri yönetimi ve yetkilendirme mekanizmaları geliştirmek.
- Farklı veri kaynaklarından gelen bilgilerin entegrasyonunu sağlamak.
- Raporlama ve analiz araçları ile kullanıcıların karar verme süreçlerini desteklemek.

Web tabanlı uygulamalar ve siteler, internet tarayıcıları üzerinden erişilebilen yazılımlar ve hizmetlerdir. Bu tür uygulamalar, kullanıcıların herhangi bir yazılım indirmesine gerek kalmadan, tarayıcı üzerinden hizmetlere erişebilmesini sağlar. Web tabanlı uygulamaların ve sitelerin başlıca avantajları şunlardır:

Erişilebilirlik: Herhangi bir cihazdan (bilgisayar, tablet, akıllı telefon) ve herhangi bir yerden internet bağlantısı ile erişim sağlanabilir.

Güncellemeler ve Bakım: Yazılım güncellemeleri ve bakım işlemleri merkezi olarak yapılabilir, bu da kullanıcıların her zaman en güncel sürümü kullanmasını sağlar.

Maliyet Etkinliği: Yazılım dağıtımı ve bakımı için düşük maliyetli çözümler sunar.

Esneklik ve Ölçeklenebilirlik: Kullanıcı sayısındaki artışa veya değişen ihtiyaçlara göre kolayca ölçeklendirilebilir.

Çok Kullanıcılı Destek: Birden fazla kullanıcının aynı anda erişim sağlayabilmesi ve ortak çalışabilmesi mümkündür.

Finansal uzlaşma sistemleri gibi kurumsal uygulamalar için web tabanlı çözümler, şirketlerin finansal süreçlerini yönetmelerini ve iş ortaklarıyla verimli bir şekilde iletişim kurmalarını kolaylaştırır. Bu bağlamda, web tabanlı uygulamaların sağladığı avantajlar, bu projenin temel bileşenlerinden biri olarak öne çıkmaktadır.

Bu çalışma kapsamında, aşağıdaki metodolojik yaklaşımlar ve yöntemler kullanılacaktır:

Literatür Taraması: Finansal uzlaşma sistemleri ve ilgili teknolojiler üzerine yapılan mevcut çalışmalar incelenecek ve teorik çerçeve oluşturulacaktır.

Sistem Analizi ve Tasarımı: Kullanıcı gereksinimlerine uygun bir sistem mimarisi tasarlanacak ve bu mimari doğrultusunda veri modelleri ve iş akışları belirlenecektir.

Uygulama Geliştirme: Sistem, modern web teknolojileri kullanılarak geliştirilecek ve modüler bir yapı üzerine inşa edilecektir.

Test ve Değerlendirme: Geliştirilen sistem, çeşitli test aşamalarından geçirilerek performans, güvenlik ve kullanıcı deneyimi açısından değerlendirilecektir.

2. Web Tabanlı Uygulamalar ve Siteler

2.1 Web Nedir?

Web WWW yani World Wide Web olarak bilinen Dünya çapında Ağın bilinen yaygın adıdır. Web ile WWW aynı kavramlardır. Web dünya üzerinde yerel bilgisayarlar üzerinden erişilebilen web sayfaları topluluğudur. Aşağıda webin icadı ve web in çalışma şekli hakkında detaylı bilgi bulacaksınız.

Günümüzde modern web tasarım projelerinin de temellerini ve standartlarını oluşturan World Wide Web web tasarımcılar ve web geliştiriciler tarafından çok yakından takip edilen bir konudur.

Web üzerinden erişilebilen dijital web sayfalarına da web sayfası denilir. Aynı alan adına tanımlı web sayfaları koleksiyonuna da web sitesi denir.

Web, dünya üzerindeki tüm insanların gelecek 30 yıldaki hayatlarını ölçeklendirilemez derecede değiştirecekti. Bugün webin gelişmesi ve arama motorlarının çok iyi algoritmalar ile çalışması sayesinde her dilde aratılan her bilgiye en kısa sürede erişim sağlanmaktadır. Dünya ucundaki bir insanın bilgi ve fikirlerine web üzerinden anında erişebileceğiniz anlamına

gelmektedir. Web günümüzde, eğlence, bilgi alma, müşteri bulma, personel bulma, para kazanma gibi bir çok konu üzerinden insanların hayatının vazgeçilmez bir parçası haline gelmiştir.

Kısaca daha da özetlemek gerekirse WWW yanın World Wide Web, internet üzerinden erişilebilen, birbirine bağlı web sayfası sistemidir. Web ile aynı manada olan WWW internet üzerinde kurulmuş birçok uygulamalardan biridir.

2.2 Web Gelişimi

Web protokolü, bilgi paylaşımı ve etkileşim konusunda görevini başarılı bir şekilde yapmaktadır. Bu sebeple Web'in yıllara nazaran gelişimi, kullanım yapısındaki önemli değişiklikler ile müthiş kırılmalara yol açmıştır. Devrim sayılabilecek yeni yetenekler kazanmıştır. Bu yeteneklerin gelişimi de hızla devam etmektedir.

Web 1.0

Web 1.0 ile internet kullanıcıları sadece HTML tabanlı olan statik web siteleri geliştirmişlerdir. Ziyaretçilerin, belirlenen içerikler ile bilgi sahibi olmaları istenilmiştir. Kısacası, Web 1.0'da, web siteleri sadece okunabilir bilgileri içermektedir. 2004 yılına kadar bu durum devam etmiş ve genel olarak tanıtım, bilgi edinme ve alışveriş yapabilmekle sınırlı kalmıştır. Kullanıcıların da teknik bilgi gerekmeden içerik üretimine dahil olmasını sağlayan Web 2.0 ile devrim sürecine girilmiştir.

Web 2.0

Bu dönemde kullanıcılar, aktif ve üretici bir rol üstlenip, içerik oluşturmaya bu içerikleri paylaşmaya, yorumlamaya başlamışlardır. Facebook ve Youtube gibi sitelere yüklenen içerikler geometrik olarak çoğalmaya başlamıştır. İçerik üretimin artması ile ihtiyaçlarına göre web ortamında uygulama ve çözümler üretilmiştir. Akıllı telefon ve cihazların da Web'e dahil olması ile Web 3.0'a geçişin adımları atılmıştır. Web 3.0'ın, internete bağlı olan tüm cihazlar, sensörler ve servisler arasında etkileşim sağlayacağı öngörülmektedir.

Web 3.0

Web 3.0 dönemi, Web'in sadece doğal dillerden oluşmayan, aynı zamanda ilgili yazılımlar tarafından anlamlandırılabilir, yorumlanabilir ve kullanılabilir bir biçimdir. Bu sayede cihazların da veriyi kolayca bulabilmesi, paylaşması ve çıkarım yapabilmesi amaçlanmaktadır.

Günümüzde de rahatlıkla bulunabileceği gibi eksikleri internetten sipariş veren bir buzdolabı, Web 2.0 döneminin alt yapısı ile çalışabilmektedir. Fakat bir buzdolabına “Bu akşam amcamlar yemeğe geliyor, ona göre hazırlık yap” şeklinde semantik bir komutun verilip, buzdolabının, kişinin amcası ve ailesinin yemek alışkanlıklarını ve gelecek kişi sayısına göre eksiklerini sipariş etmesi ise Web 3.0 dönemi ait olacaktır.

Web 4.0

Bu teknolojilerde saniyede 100 Gigabit bağlantı ve bant genişliğine sahip ağlar ile depolama sistemlerinin tamamen Web ortamına taşındığı, günlük kullanımdaki neredeyse tüm eşyaların yapay bir zekaya sahip olup ağlar üzerinden birbirine bağlandığı bir dönemden bahsedilmektedir. İşletim sistemlerinin bulut teknolojileri sayesinde Web’e taşındığı, herkesin ve her şeyin kendisine has internet kimlik numarası ile ağa bağlı bulunduğu bir dönem olarak tarif edilmektedir.

2.3 Neden Web Tabanlı?

Web tabanlı uygulamalar ve siteler, geleneksel masaüstü uygulamalarına göre birçok avantaja sahiptir:

Platform Bağımsızlığı: Web tabanlı uygulamalar, farklı işletim sistemleri ve cihazlar üzerinde çalışabilir. Kullanıcılar, bilgisayarlarının işletim sistemine veya cihaz türüne bakılmaksızın web tarayıcıları aracılığıyla erişim sağlayabilirler.

Güncelleme Kolaylığı: Web tabanlı uygulamalar, kullanıcının herhangi bir güncelleme yapmasına gerek kalmadan sunucu tarafında güncellenebilir. Bu, kullanıcıların yeni özelliklere veya düzeltmelere hızla erişebilmesini sağlar.

Dağıtım Kolaylığı: Web tabanlı uygulamalar, kullanıcılara herhangi bir kurulum gerektirmeden erişim sağlar. Bu, kullanıcıların uygulamayı hızla kullanmaya başlayabilmelerini sağlar.

Kullanıcı Tabanlı Veri Depolama: Web tabanlı uygulamalar, sunucu tabanlı veri depolama kullanarak kullanıcıların verilerini güvenli bir şekilde saklar. Bu, kullanıcıların farklı cihazlardan erişim sağlamasını kolaylaştırır ve veri kaybını önler.

Geniş Kitle Erişimi: Web tabanlı uygulamalar, internete erişimi olan herkes tarafından kullanılabilir. Bu, geniş bir kitleye hitap etme potansiyeli sunar.

Bu avantajlar, web tabanlı uygulamaların ve sitelerin giderek daha popüler hale gelmesine ve birçok alanda kullanılmasına neden olmuştur.

2.4 Web Sitesi

İnternet, kişilerin değişik amaçlarla ve içerikte karşılıklı iletişim kurmalarını, bilgi alıp vermelerini sağlayan ortak bir haberleşme altyapısıdır. İnternet, çok sayıda iletişim ağından (network) oluşan, güvenilir, sıralı ve uçtan uca bilgi transferini sağlamak için TCP(Transmission Control Protocol)/IP(Internet Protocol) protokolünü kullanan şebekeler topluluğudur. TCP/IP olarak adlandırılan bu ortak dil, yüksek kapasiteli telefon hatları üzerinden bilgisayarların birbirleri ile iletişim kurabilmelerine ve karşılıklı bilgi alışverişinde bulunabilmelerine imkan tanır(Erol, 2001,57). İnternet ortamında ticari faaliyetler bir web sunucusuna bağlı web siteleri aracılığıyla gerçekleştirilir. Web siteleri, ürün tanıtımı, pazarlanması ile firmanın başta müşteriler ve ortaklar olmak üzere tüm işletme ilgililerine bilgi iletimine imkan sağlayan sanal ortamlardır. Web sitesi tasarımında sürekli yeni yazılım ve donanım araçları ortaya çıkmaktadır. Web sitelerinin sayısının günden güne artması ve şiddetlenen rekabetin sonucu olarak, ürün/hizmet farklılaştırma, sürekli yenilik yaratma, uygun fiyat, tüketicilere fayda sağlama, zengin ve Web Sitesi Maliyetlerinin Muhasebeleştirilmesi 13 güvenilir içerik ile kaliteli tasarım elektronik ticarete temel rekabet faktörleri halini almıştır.(Erdal, 2003,36) . Bir web sitesi, firma ile müşteriler arasında mal, hizmet ve bilgi alışverişinin internet ortamında gerçekleştirilmesini sağlar. Firmalar web siteleri aracılığıyla mal ve hizmetlerini tanıtmakta, birbirlerini tanımakta ve iletişimi geçerek uygun bir şekilde iş yapma olanakları elde etmektedirler. Web siteleri, otomasyonlaşmış sistemlerin ortaklaşa iş yapılan birimlere entegrasyonu ile ürün ve hizmetler ile bilgilerin satış, kullanım ve paylaşımına imkan sağlanmaktadır(Çak, 2002,39) Firmalarda internet teknolojisinden yararlanma son yıllarda üç aşamalı bir gelişme göstermektedir. Birinci olarak, internet, www sayfalarının ve diğer ağların kullanımına dayalı bir yapılanmaya gidilmekte, ikinci olarak firmalar kendi iç işleyişleri ya da işlemleri için bir içsel ağ sistemi oluşturmakta, üçüncü olarak ise tedarikçiler ve diğer firmalara bağlanmak amacıyla dışsal ağ düzeni oluşturmaktadır(Kepenek, 1999, 59-60). Günümüzde bireysel ve kurumsal internet kullanımındaki artışın sonucu olarak web siteleri daha işlevsel ve müşteri odaklı hale gelmiştir. Reklam çalışmalarına benzer bir şekilde teknik, sanatsal ve bilimsel yönleri olan bir çalışma ile oluşturulan web sitesi ticari başarıyı doğrudan etkilemektedir. Web sitesinin hazırlanmasında erişilebilirlik hızı, iletişim hızı, sayfalar arasında aradığını kolaylıkla bulabilme ve ilgili diğer

sayfalara kolay erişebilme gibi teknik özellikler, web sayfasının içeriği, web sayfasının formu, web sayfasının görsel olarak tüketicilerin dikkatini çekecek bir sayfa olması önemli karar başlıklarıdır(İTO, 2001,9).

3. FİNANSAL UZLAŞMA

3.1 Finansal uzlaşma Nedir?

Şirketler arası yada kişiler arası bazı konularda ortak nokta bulunmasına denir. Bunlar alacak verecek yada yıllık izin gibi konularda gerçekleşebilir. Şirketler genelde yıl sonu yaklaştığında her alanda kişilerle yada diğer firmalarla mutabık olmak isterler. Çalışanların yıllık izinleri de buna dahildir. Yıllık izinleri ile de anlaşma sağlanması gerekir. Aksi takdirde işçinin hesapladığı gün sayısı ile iş verenin hesapladığı gün sayısı arasında fark oluşabilir. Bu gibi durumlar zaman içerisinde sorunlara yol açabilir.

Öncelikle sistem üzerinden göndermek istenilen uzlaşma seçeneğini seçilmelidir. Ardından cari listenize açıklama bilgilerini girerek finansal uzlaşma göndermek istediğiniz firmaları ve şahısları seçilmelidir. Ayrıca farklı bir form tasarımı ya da mail kullanmak istediğiniz zaman yükleyerek kullanabilirsiniz. Gönderim yapacağınız kişileri excel üzerinden kopyala yapıştır özelliği kullanılarak kolayca aktarımı yapılabilmektedir.

Finansal uzlaşma yapmak istediğiniz kişileri ve firmaları seçerek geçerli olan tarihi girilmelidir. Böylece, kayıt edilen uzlaşma formlarını istenildiği zaman görüntülenebilmektedir. Finansal uzlaşma formu türüne göre yetkilendirilmeye olanak tanımaktadır. Örneğin finansal hesap uzlaşması formunu farklı bir departman üzerindeki BA/BS formunu başka bildirimlere göndermek üzere tasarlamayı sağlar.

3.2 Nasıl Finansal uzlaşma Yapılır?

Finansal uzlaşma süreci genel olarak şu adımları içerir:

- Veri Toplama: İlgili mali verilerin dijital formatta toplanması.
- Veri Karşılaştırma: İşletmenin kendi kayıtları ile karşı tarafın kayıtlarının otomatik olarak karşılaştırılması.
- Hata ve Farkların Belirlenmesi: Kayıtlar arasındaki farkların ve hataların tespit edilmesi.

- Düzeltme ve Onay: Tespit edilen hataların düzeltilmesi ve her iki tarafın uzlaşmayı onaylaması.
- Kayıt ve Raporlama: Uzlaşmanın dijital olarak kayıt altına alınması ve raporlanması.

3.3 Finansal hesap uzlaşması nedir?

Finansal hesap uzlaşması uzlaşmayı şirketlerin karşılıklı olarak mal veya hizmet alım satımlarında oluşacak olan bakiyeyi takip etmek amaçlı yapılmaktadır. Şirketler karşılıklı olarak mutabık kaldıkları finansal hesap uzlaşması formunu kaşe ve imzalı olarak birbirlerine göndermektedir. Bu sayede, formlar ile ürün ve malım alı satım işlemleri gerçekleştirilir. Böylece formlar üzerinde yapılan işlemlerle hesaplarda karşılıklı olarak uzlaşma sağlanmış olur.

Finansal hesap uzlaşması uzlaşma formu hazırlama için Excel kullanılır. Excel programında hazırlanan formu aynı zamanda faks, telefon ya da e posta üzerinden gönderebilirsiniz. Fakat bu gibi işlemlerle hata yapma riski daha yüksektir. Zaman kaybettiren bir işlem oluşmasına yol açmaktadır. Dolayısıyla, yapılan işlemleri elektronik ortam üzerine taşımak zaman kaybını ortadan kaldırmaktadır. Ayrıca hata yapma olasılığını en aza indirmektedir. Bu sebepten dolayı bir entegratör firma ile çalışmaya başlayarak otomatik mütabakatlar

3.4 BA/BS Uzlaşması Nedir?

Ba ve Bs formları, Türkiye'de vergi mükelleflerinin düzenlediği ve Gelir İdaresi Başkanlığı'na gönderdiği bildirim formlarıdır.

- Ba Formu (Bildirim Alacak Formu): Vergi mükellefinin belirli bir dönem içinde yaptığı alışları gösterir.
- Bs Formu (Bildirim Satış Formu): Vergi mükellefinin belirli bir dönem içinde yaptığı satışları gösterir.

BA/BS uzlaşması mükellefler vergi dairesine BA/BS veri formlarını göndermeden önce müşterileri ve karşı tarafla uzlaşma yapmalarına denir. Gelir Dairesi Başkanlığı'nın önemli bir parçasıdır. Maliye Bakanlığı'na bağlıdır. BA kısaltması 5 Bin TL üzerin yapılan alımlar, BS kısaltmasının ise 5 Bin TL üzeri satışlar anlamına gelir. Hizmet veya mal satışları bildirimlerinde kullanılan bir formdur.

Bu formlar her ay düzenli olarak bilanço esasına göre defter tutan ticari işletmelerin veya tüzel kişiliklerin ayın 30'una kadar bildirilmek zorunda olduğu resmi bir form türüdür. Malın alış ve alış iadesi, demir başlar, satış ve satış iadesi, fatura edilmiş belgeler ve giderlerin tamamını kapsamaktadır. Bu belgeler K.D.V hariç olarak 5000 TL'yi geçen fatura adetleri ve fatura toplamaları bildirilmelidir.

01.06.2024 tarihinden itibaren ise elektronik ortamda düzenlenen faturalar Ba-Bs bildirimlerine konu olmayacağından dolayı 5 Bin TL'yi geçen elektronik olarak düzenlen hem alım hem de satım faturaları Ba-Bs formlarında bildirilmeyecektir. Ancak kağıt ortamında düzenlenen faturalar, Gider pusulaları, ödenen kiralar, yurtdışına yapılan ödemeleri vb. benzeri sınırları geçen makbuzlar, faturalar veya evraklar Ba-Bs formlarında beyan edilmek zorundadırlar.

3.5 Finansal Uzlaşma Yöntemini Kimler Kullanabilir?

Ülkemizde 20 yıldır sunulan tescilli elektronik uzlaşma hizmetini sunan EDM Bilişim Yetkililerine göre; Finansal uzlaşma yönetimini kullanmak için, BA/BA form beyanlarını, Finansal hesap uzlaşması uzlaşmalarını dijital ortamda kullanmak isteyen bütün firmalar ve şirketler kullanabilmektedir. Bunun için yalnızca bilanço esasına göre defter tutmak yeterlidir. Tedarikçiler, bayiler, iş ortakları çok olan şirketler ve partnerler, arşivleme, zamandan tasarruf etmek isteyenler kullanabilir. Aynı zamanda, para tasarrufu yapmak isteyenler ile maliyetini düşürmek isteyen bütün firmalar finansal uzlaşma yönetimini kullanabilir.

3.6 Finansal Uzlaşma Yararları Nelerdir?

Firma ve çalışanlara finansal uzlaşma üzerinden işlem yaparken kağıt israfını ortadan kaldırmaktadır. Telefon görüşmeler, faks çıktıları, arşivleme, kargo gönderimi gibi bütün operasyonel işlemlerini yapmanızı sağlamaktadır. Finansal uzlaşmanın yararları arasında bulunanları şu şekilde sıralamak mümkündür.

- Finansal uzlaşma uygulaması ile muhasebe verileriniz daha tutarlı olacaktır.
- Firmaların verimlilik sağlayarak zamandan tasarruf etmesini sağlar.
- Uzlaşma sayıları ile mail gönderilerinin sınırı yoktur.
- Takiplerin tamamı elektronik ortamda yapıldığından hataların önüne geçilmiş olur.
- Yapılan bütün işlemler elektronik ortamda yapıldığı için kağıt masrafı ile dosya masrafı olmaz.

- Uzlaşmalar gönderim ve sonuçlanma süreçlerinde tek platform üzerinden takip edilebilir. Dolayısıyla, platform üzerinde iletişim kalitesi yüksektir.
- Ekstreler arası farklar otomatik gösterildiğinden zaman kaybı %100 biter.

3.7 Bu Projenin Etkisi

ReconciliationBackendProject'in etkileri şunlar olabilir:

- İş Süreçlerinin İyileştirilmesi: Proje, işletmelerin uzlaşma süreçlerini hızlandırarak ve otomatikleştirerek iş süreçlerini iyileştirir.
- Hata Oranının Azaltılması: Manuel işlemlerden kaynaklanan hata oranını azaltır.
- Verimlilik Artışı: Daha hızlı ve verimli uzlaşma süreçleri sağlar.
- Maliyet Azaltma: İş gücü ve zaman tasarrufu sağlayarak maliyetleri düşürür.
- Güvenlik ve İzlenebilirlik: Verilerin güvenli ve izlenebilir şekilde saklanması sağlar.

4. Literatür Taraması

4.1 Finansal uzlaşma Sistemleri: Tarihçe ve Gelişim

Finansal uzlaşma sistemleri, şirketler arasında finansal bilgilerin elektronik ortamda doğrulanmasını sağlayan sistemlerdir. İlk olarak 1990'lı yılların sonlarında elektronik veri değişimi (EDI) sistemlerinin bir parçası olarak ortaya çıkan finansal uzlaşma sistemleri, internetin ve dijital teknolojilerin gelişmesiyle birlikte hızla yaygınlaşmıştır. Başlangıçta sadece büyük ölçekli şirketler ve uluslararası ticaret yapan firmalar tarafından kullanılan bu sistemler, günümüzde her ölçekten şirketin kolayca erişebileceği ve kullanabileceği hale gelmiştir.

Finansal uzlaşma sistemlerinin gelişimi, şirketlerin manuel uzlaşma süreçlerindeki verimsizlikleri ve hataları azaltma ihtiyacından kaynaklanmaktadır. Geleneksel uzlaşma süreçleri, zaman alıcı ve hata yapma olasılığı yüksek işlemler içerir. Finansal uzlaşma sistemleri ise, bu süreçleri otomatikleştirerek daha hızlı, verimli ve güvenli bir uzlaşma sağlamaktadır. Bu sistemlerin gelişimi, özellikle bulut bilişim ve veri analitiği teknolojilerindeki ilerlemelerle desteklenmiştir.

4.2 Mevcut Çalışmalar ve Yöntemler

Literatürde, finansal uzlaşma sistemlerinin farklı sektörlerdeki uygulamalarını ve sağladığı avantajları ele alan birçok çalışma bulunmaktadır. Örneğin, Yazar (2019), finansal uzlaşma sistemlerinin muhasebe süreçlerindeki hata oranını %30 oranında azalttığını ve işlem sürelerini yarı yarıya kısalttığını belirtmektedir. Ayrıca, finansal uzlaşma sistemlerinin kullanıcı dostu arayüzleri ve otomatik veri doğrulama özellikleri sayesinde kullanıcı memnuniyetini artırdığı görülmektedir (Yazar, 2018).

Başka bir çalışma, Yazar (2016), finansal uzlaşma sistemlerinin güvenlik açıklarını ve bu açıkların nasıl giderilebileceğini analiz etmiştir. Bu çalışmada, güvenlik protokollerinin ve şifreleme tekniklerinin kullanılmasıyla finansal uzlaşma sistemlerinin daha güvenli hale getirilebileceği vurgulanmaktadır. Benzer şekilde, Yazar (2017), ilişkisel veri tabanı yönetim sistemlerinin (RDBMS) finansal uzlaşma sistemlerindeki veri bütünlüğünü ve güvenliğini sağlama konusundaki etkinliğini incelemiştir.

Literatürde, katmanlı mimari kullanılarak geliştirilen finansal uzlaşma sistemleri de incelenmiştir. Katmanlı mimari, yazılım projelerinin modüler, sürdürülebilir ve bakımının kolay olmasını sağlar. Örneğin, Yazar (2017), katmanlı mimari kullanılarak geliştirilen bir finansal yönetim uygulamasının, modüler yapısı sayesinde yeni özelliklerin kolayca eklenebildiğini ve mevcut özelliklerin güncellenebildiğini belirtmektedir. Ayrıca, katmanlı mimarinin, yazılım geliştirme süreçlerinde karşılaşılan karmaşıklıkları azaltarak projelerin daha hızlı tamamlanmasını sağladığı da vurgulanmaktadır.

4.3 Teorik Çerçeve ve Temel Kavramlar

Finansal uzlaşma sistemlerinin geliştirilmesi ve uygulanması sürecinde, çeşitli teorik çerçeveler ve temel kavramlar kullanılmaktadır. Bu kavramlar, sistemlerin tasarımı, veri yönetimi ve kullanıcı etkileşimi gibi konularda önemli rehberlik sağlar.

4.3.1 Katmanlı Mimari

Katmanlı mimari, yazılım geliştirme sürecinde uygulamanın farklı katmanlara ayrılarak her bir katmanın belirli bir işlevi yerine getirdiği bir tasarım modelidir. Bu mimari model, yazılım projelerinin modüler, sürdürülebilir ve bakımının kolay olmasını sağlar. Katmanlı mimarinin

temel avantajları arasında kod tekrarının azaltılması, bileşenler arasındaki bağımlılıkların en aza indirilmesi ve uygulamanın esnekliğinin artırılması yer almaktadır.

4.3.2 Veri Tabanı Yönetim Sistemleri (VTYS)

Veri tabanı yönetim sistemleri (VTYS), büyük miktarda verinin saklanması, yönetilmesi ve işlenmesi için kullanılan yazılımlardır. VTYS, veri bütünlüğünü ve güvenliğini sağlar, veri erişimini kolaylaştırır ve veri yönetimi süreçlerini otomatikleştirir. Finansal uzlaşma sistemlerinde, kullanıcı bilgileri, finansal veriler ve uzlaşma işlemleri gibi kritik verilerin güvenli bir şekilde yönetilmesi için VTYS kullanımı oldukça önemlidir.

4.3.3 Kullanıcı Yetkilendirme (Rol Bazlı Yapı) ve E-Mail Parametreleri

Finansal uzlaşma sistemlerinde, kullanıcıların yetkilendirilmesi ve e-mail parametrelerinin yönetimi, sistem güvenliği ve işlevselliği açısından kritik öneme sahiptir. Kullanıcı yetkilendirme mekanizmaları, kullanıcıların sistemde hangi işlemleri gerçekleştirebileceğini belirler. E-mail parametreleri yönetimi ise, sistemin kullanıcılara otomatik bildirimler göndermesini sağlar ve uzlaşma süreçlerinin etkin bir şekilde yürütülmesini destekler (Yazar, 2019).

Bu teorik çerçeve ve temel kavramlar, finansal uzlaşma sistemlerinin geliştirilmesi ve uygulanmasında önemli bir rol oynamaktadır. Literatürde bu kavramların ve teorik çerçevelerin farklı projelerde nasıl kullanıldığı ve sağladığı avantajlar üzerine yapılan birçok çalışma bulunmaktadır.

Rol bazlı yapı (Role-Based Access Control - RBAC), kullanıcılara belirli roller atayarak erişim kontrolünü yönetme yöntemidir. Her rol, belirli erişim izinlerine sahiptir ve kullanıcılar sadece atanmış rolleri doğrultusunda sistemde işlem yapabilirler. Bu yöntem, güvenliği artırır ve yönetimi kolaylaştırır.

Dependency Injection yöntemi ile sınıfların bağımlılıklarını dışarıdan almasını sağlayarak, kodun daha esnek, yeniden kullanılabilir ve test edilebilir olmasını sağlar..

Ardından JWT alt yapısı kurulur. JWT (JSON WEB TOKEN), kullanıcıların kimlik doğrulaması ve yetkilendirilmesi için kullanılır.

Ardından token oluşturma altyapısı hazırlanır. En son olarak güvenlik ayarları yapılır

```

public AccessToken CreateToken(User user, List<OperationClaim> operationClaims, int companyId)
{
    _accessTokenExpiration = DateTime.Now.AddMinutes(_tokenOptions.AccessTokenExpiration); //Token
    çağırıldığından itibaren dakika olarak ekliyoruz.
    var securityKey = SecurityKeyHelper.CreateSecurityKey(_tokenOptions.SecurityKey);
    var signingCredentials = SigningCredentialsHelper.CreateSigningCredentials(securityKey);
    var jwt = CreateJwtSecurityToken(_tokenOptions, user, signingCredentials, operationClaims,
        companyId);
    var jwtSecurityTokenHandler = new JwtSecurityTokenHandler();
    var token = jwtSecurityTokenHandler.WriteToken(jwt);

    return new AccessToken
    {
        Token = token,
        Expiration = _accessTokenExpiration,
        CompanyId = companyId
    };
}

```

Projede otomatik e-posta gönderim işlemimiz mevcut. SMTP ayarlarının yapılandırılması, e-posta gönderim servisi ve DI (Dependency Injection) ile entegrasyonu adım adım ele alınmıştır. Bu yapı ile kullanıcılar, bildirim, doğrulama ve diğer iletişim amaçları için otomatik olarak e-posta alabilirler.

```

builder.Services
    .AddFluentEmail("info@admin.com") //burası bizim mail adresimiz
    .AddSmtpSender("localhost", 25); //burası SMTP ile bağlantı yerimiz

```

```

void SendConfirmEmail(User user)
{
    string subject = "Kullanıcı Kayıt Onay Maili";
    string body = "Kullanıcınız sisteme kayıt oldu. Kaydınızı tamamlamak için aşağıdaki linke tıklamanız gerekmektedir.";
    string link = "https://localhost:7020/api/Auth/confirmuser?value=" + user.MailConfirmValue;
    string linkDescription = "Kaydı Onaylamak için Tıklayın";

    var mailTemplate = _mailTemplateService.GetByTemplateName("Kayıt", 6);
    string templateBody = mailTemplate.Data.Value;
    templateBody = templateBody.Replace("{{title}}", subject);
    templateBody = templateBody.Replace("{{message}}", body);
    templateBody = templateBody.Replace("{{link}}", link);
    templateBody = templateBody.Replace("{{linkDescription}}", linkDescription);

    var mailParameter = _mailParameterService.Get(6);
    SendMailDto sendMailDto = new SendMailDto()
    {
        mailParameter = mailParameter.Data,
        email = user.Email,
        subject = "Kullanıcı Kayıt Onay Maili",
        body = templateBody
    };

    _mailService.SendMail(sendMailDto);

    user.MailConfirmDate = DateTime.Now;
    _userService.Update(user);
}

```

```
public void SendMail(SendMailDto sendMailDto) //otomatik mail göndermek için bu işlemi yapıyoruz.
{
    //FluentEmail.To(sendMailDto.email).Subject(sendMailDto.subject).Body(sendMailDto.body).Send();
    using (MailMessage mail = new MailMessage())
    {
        mail.From = new MailAddress(sendMailDto.mailParameter.Email);
        mail.To.Add(sendMailDto.email);
        mail.Subject = sendMailDto.subject;
        mail.Body = sendMailDto.body;
        mail.IsBodyHtml = true;
        //mail.Attachments.Add();

        using (SmtpClient smtp = new SmtpClient(sendMailDto.mailParameter.SMTP))
        {
            smtp.UseDefaultCredentials = false;
            smtp.Credentials = new NetworkCredential(sendMailDto.mailParameter.Email, sendMailDto.mailParameter.Password);
            smtp.EnableSsl = sendMailDto.mailParameter.SSL;
            //smtp.Port = sendMailDto.mailParameter.Port;

            smtp.Send(mail);
        }
    }
}
```



s.ior@hotmail.com

Kime: Siz



1.05.2024 Çar 02:48

Kullanıcı Kayıt Onay Maili

Kullanıcınız sisteme kayıt oldu. Kaydınızı tamamlamak için aşağıdaki linke tıklamanız gerekmektedir.

[Kaydı Onaylamak için Tıklayın](#)

5. SİSTEM TASARIMI VE MİMARİSİ

5.1 Uygulama Katmanı (Backend)

Back-end (arka uç), kullanıcının bir internet sayfasında veya bir uygulamada göremediği ama arka tarafta uygulamanın düzgün çalışmasını sağlayan kaynak kodlarından ve veritabanlarından oluşan bölümdür. Ayrıca, uygulamanın “sunucu tarafı” olarak da adlandırılır.

Back-end geliřtiriciler, bir veritabanının ve uygulamanın birbiriyle iletiřim kurmasına, uygulamanın sorunsuz alıřmasına izin veren kodları oluřturur. Genel olarak, veritabanları, sunucular ve uygulamalar dahil olmak zere bir web sitesinin arka ucuyla ilgilenir geliřtirme ve kullanıcının gremediėi kısımların bakımlarını yapar.

5.2 Veri Tabanı Katmanı

Veri ok farklı formlarda karřımıza ıkabilir. Rakamlardan, yazıdan, bitlerden ya da grsellerden oluřabilir. Elektronik hafızada saklanabildiėi gibi kâğıt zerinde yazılı olarak da tutulabilir.

“Tek para bilgi” anlamına gelen datum kelimesinin oėulu olan data gnmzde bilgi teknoloėisi alanında en ok kullanılan terimlerden biri hline gelmiřtir. Bilgisayarlarda veri bařka bir forma, verimli bir harekete ya da iřleme dnřebilen bilgi anlamına gelmektedir ve deėiřebilir bir forma sahiptir.

Veri tabanı, yani database ise dzenli bir řekilde toplanmıř veriye verilen isimdir. Burada dzenli kelimesine dikkat ekmek gerekir, veri tabanındaki verilerin kolayca ulařılabilir ve ynetilebilir olması gerekir.

Veri tabanındaki veriler; tablolar, sıralar, kolonlar ve indeks olarak dzenlenebilir. Bu tamamen verinin cinsine ve kullanıcıların tercihlerine baėlıdır ve asıl amacı ilgili bilginin kolay bir řekilde bulunmasıdır. nk veri tabanlarının ana amacı byk miktarda verinin kolay bir řekilde ynetilip kullanılacaėı zaman hızla ulařılacak řekilde depolanabiliyor olmasıdır.

Gnmzde dnyada pek ok dinamik web site bulunmaktadır. Bu siteler veri tabanları ile ynetilir.

Kullanıcı bilgileri, finansal veriler ve uzlařma iřlemleri gibi kritik veriler bu katmanda depolanır. Veri tabanı ynetim sistemi (VTYS) olarak iliřkisel veri tabanları (rneėin, Microsoft SQL Server, MySQL) kullanılır. En yaygın olarak bilinen veri tabanı dili SQL’dir; ancak, Yapılandırılmıř Sorgu Dili’nin (SQL) farklı eřitleri vardır. Her bir SQL tr, SQL dil yapısında zıtlıklara sahiptir ve belirli bir veri tabanı tr ile kullanılmak zere tasarlanmıřtır.

5.3 Katmanlı Mimari

Her bir katmanın kendine ait bir işi yapması ve bu sayede tek bir katman üzerinde tüm işlemlerin yapılıp bir karmaşanın oluşmasını engellemek için katmanlı mimari ortaya çıkmıştır.

5.3.1 Neden Katmanlı Mimariye İhtiyaç Duyuyoruz?

Bu tez de bizim burada mevcut bir projeye site yapıyoruz. Bizim bu sitede işlemlere ihtiyacımız olacak. En basitinden create, update, delete, read (ekleme,güncelleme,silme,okuma) işlemleri yapacağız.(işlem gönderiyorum, işlem alıyorum, işlem gönderiyorum alıyorum.....) Bu işlemler için bazı kontroller gerçekleştireceğiz. Validation kontrol gibi Validation kontrolünde de mesela ben bir uzlaşma eklemek istiyorsam örnek veriyorum 300 TL'nin altındakileri cari seçilmemişse uzlaşma listesine ekleme, cari oluştururken mesela diyeceğiz ki cari ismi 3 karakterden az olanların ismini oluşturma bunlara işte Validation kontrol deniyor. İşte şimdi bu tür işlemleri eğer katmanlı mimari oluşturmazsam WebApi kısmının Controllers'ında tek tek yazmam gerekecek kod kalabalığı oluşacak elimizde bir karmaşa oluşacak. Katmanlı Mimari işte bizi bu kalabalıktan kurtarıyor.

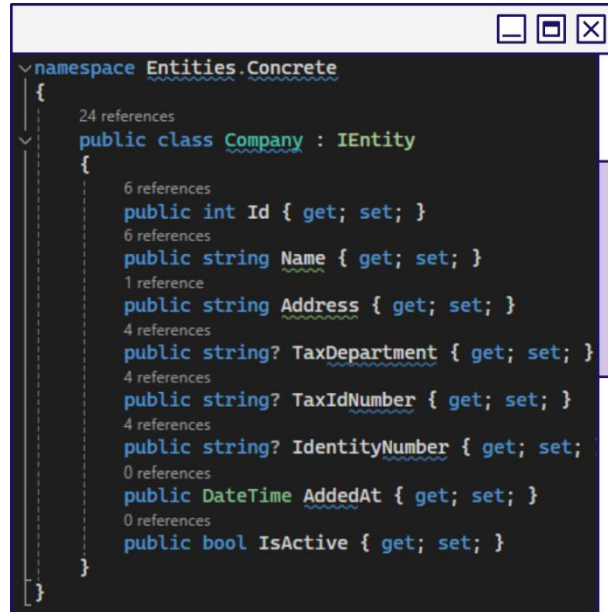
Ben bir kayıt işlemi yapmak istersem bunu direkt gidip business katmanına gönderiyor. Business katmanım bu kayıt için gerekli olan validation, transection, loglama gibi özellikleri konte ediyor eğer herhangi bir sıkıntı yoksa DataAccess Katmanına gidiyor. DataAccess katmanı artık kendi olan tek görev bu dataları işlemektir. Hemen gidiyor Database'e işlemi yaptırıyor, sonucunu alıyor, Business katmanına geri dönüyor. Business katmanı sonuçta eğer hiçbir sıkıntı bulmazsa WebApi'ye cevabını gönderiyor. Böyle WebApi katmanımız tertemiz oldu çünkü her katman kendine ait işi yapıyor.

Entities katmanı bizim classlarımızı, tablolarımızı tutuyor. Core katmanı ise tüm projeye hizmet eden ortak yapıları tutuyor. Mesela Validation için bir Validation tool yazacağız Core katmanı tutacak ve Business katmanına gönderecek. DataAccess katmanı için, Entities katmanı için, WebApi katmanı için başka bir özellikler tutacak. Kısaca Core katmanı hizmet katmanıdır. Tüm projede kullanabileceğimiz özellikleri yazarız sonra Core katmanına atarız Core katmanı da tüm projeye hizmet eder.

Her proje, her katman kendine ait bir işi yapıyor, başka bir iş yapmıyor. Bu sayede bir hata alsak veya işlemi irdelemek istesek hangi katmana bakacağımızı ve nasıl yapacağımızı rahat bulabiliriz.

5.3.2 Entities Katmanı

Entities katmanı, genellikle veritabanı tablolarına karşılık gelen sınıfların ve nesnelerin tanımlandığı yerdir. Bu katmanda veri taşıyan nesneler ve veri yapısı tanımları bulunur. Örneğin, bu proje de “Kullanıcılar”, “Şirketler” veya “Yetkiler” gibi nesnelerin sınıfları Entities katmanında yer alabilir.

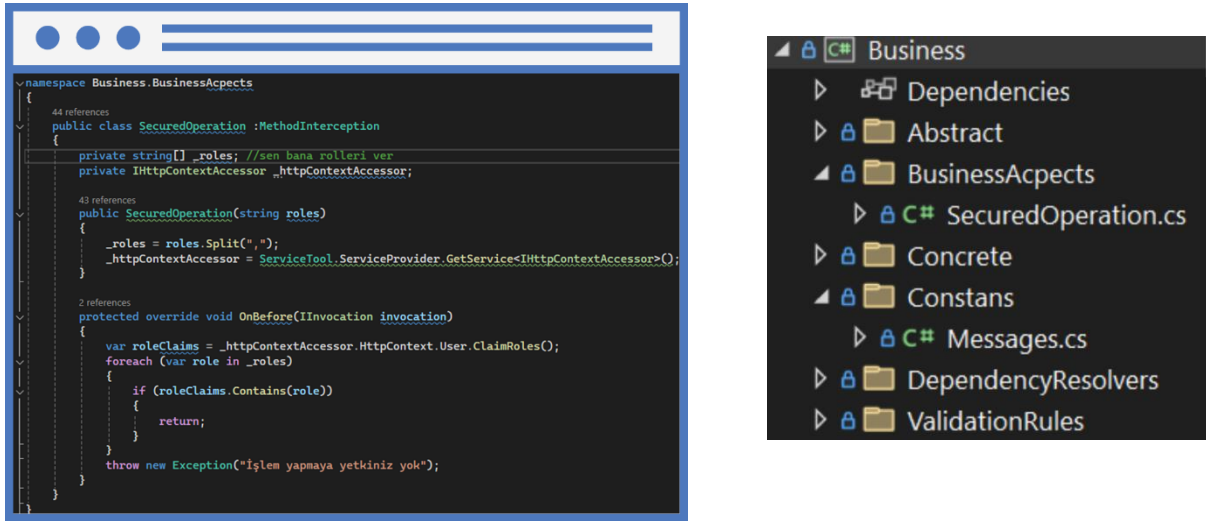


5.3.3 Core Katmanı

Core katmanı, genel işlevleri ve ortak kod parçalarını içeren temel bir katmandır. Bu katmanda, uygulamanın çekirdek işlevleri ve iş mantığı yer alır. Genellikle hata işleme, günlükleme, yetkilendirme, kimlik doğrulama ve yardımcı sınıflar gibi genel hizmetler bu katmanda bulunur.

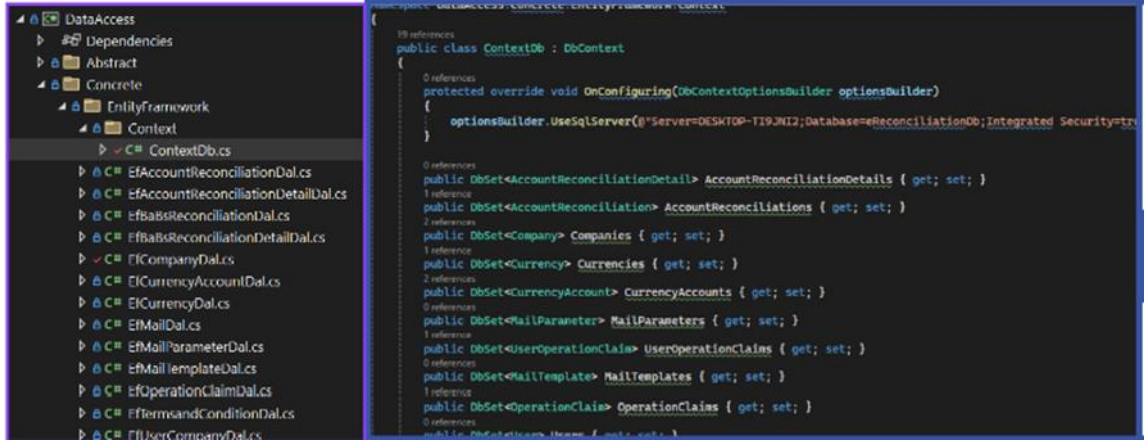
5.3.4 Business Katmanı

Verilerin işlenmesi ve iş kurallarının uygulanması bu katmanda gerçekleşir ve data acces katmanına iletilir. Bu katmanda iş kuralları veri doğrulama, iş süreci, yetkilendirme ve ilişki kuralları gibi çeşitli işlemleri gerçekleştirebiliyoruz. Bu kurallar, uygulamanın iş süreçlerini, veri doğrulamalarını ve iş mantığını kapsar. Böylece uygulamanın doğru, tutarlı ve güvenli bir şekilde çalışması sağlanır. Örneğin projem de bir uzlaşma işlemi yapılacaksa gerekli doğrulamalar, kurallar ve mesajlar bu katmanda kontrol edilir ve uygulanır.



5.3.5 Data Acces Katmanı

Data Access katmanı, Uygulamamın veri tabanı ile nasıl iletişim kuracağını tanımlayan sınıf "ContextDb" sınıfı oluşturdum ve bu sınıf Entity Framework Core (EF Core) ile çalışır. Entity Framework Core, .NET uygulamaları için bir ORM (Object-Relational Mapper) aracıdır, yani nesneleri (class) veritabanı tablolarına dönüştürür ve bu tablolar üzerinde işlem yapmanızı sağlar. CRUD (Create, Read, Update, Delete) işlemleri bu katmanda yapılır. Business katmanından gelen veri işleme taleplerini alır ve veri tabanı ile etkileşim kurarak gerekli işlemleri gerçekleştirir. Örneğin, yeni bir kullanıcı kayıt olduğunda, bu bilgi data acces katmanında veri tabanına eklenir.



5.3.6 Presentation Layer (Sunum Katmanı)

Bu katman kullanıcının uygulama ile iletişim kurduğu yerdir. Kullanıcının giriş yapması veya kayıt olması gibi işlemler. Kullanıcıdan gelen HTTP isteklerini alır, iş business katmanına iletir ve sonucu kullanıcıya döner. Genellikle bu katmanda kullanıcı arayüzü, kullanıcı girişi doğrulama ve kullanıcı etkileşimiyle ilgili işlevler bulunur. Web API (Web Application Programming Interface), HTTP üzerinden iletişim kurarak istemcilerin (örneğin web tarayıcıları veya mobil uygulamalar) bir sunucudaki kaynaklara (veritabanı, dosya, işlem vb.) erişmesini sağlayan bir yazılım arayüzüdür.

Web API, Controller ve HTTP İstekleri Arasındaki İlişki

Web API'lar, Controller sınıfları tarafından işlenen HTTP isteklerini kullanarak bir sunucudaki kaynaklara erişim sağlarlar. Bir HTTP isteği belirli bir URL' ye gönderilir ve bu istek, uygun Controller sınıfı tarafından alınarak işlenir. Controller, gerekli iş mantığını yürütür ve sonuçları uygun HTTP yanıtı olarak geri gönderir. Bu, istemcilerin (örneğin web tarayıcıları veya mobil uygulamaların) sunucu tarafındaki kaynaklara erişmesini sağlar.

Controller Nedir?

Controller, bir Web API uygulamasının kullanıcı isteklerini alıp işleyen bileşenidir. Controller, HTTP isteklerini alır, uygun iş mantığını yürütür ve sonuçları HTTP yanıtları olarak geri gönderir. Her bir Controller sınıfı, belirli bir API rotası temsil eder. HTTP istekleri, istemci tarafından sunucuya gönderilen mesajlardır.

```
builder.Services
    .AddFluentEmail("info@admin.com") //burası bizim mail adresimiz
    .AddSmtpSender("localhost", 25); //burası SMTP ile bağlantı yerim

var app = builder.Build();

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseCors(builder => builder.WithOrigins("https://localhost:7020",
app.UseHttpsRedirection();

app.UseAuthentication(); //giriş
app.UseAuthorization(); //yetkilendirme
```

5.3.7 Web Api

Web API (Application Programming Interface), iki uygulamanın veya sistemin birbirleriyle iletişim kurmasını sağlayan bir ara yüzüdür. Web API katmanı, özellikle web tabanlı uygulamalarda, farklı uygulamalar veya sistemler arasında veri alışverişini ve işlevselliği sağlamada kritik bir rol oynar. Bu katman, genellikle HTTP protokolü kullanarak istemciler ve sunucular arasında iletişimi sağlar.

5.4 Veri Tabanı Tasarımı

Veritabanı, kullanıcı bilgileri, yetkilendirme bilgileri, şirket bilgileri, hesap uzlaşmaları ve diğer ilgili verileri saklamak için çeşitli tablolar içerir. Her tablo, belirli bir veri kümesini saklamak ve yönetmek için tasarlanmıştır.

Tablo Adı – Türkçe Karşılığı	İşlevi
Users (Kullanıcılar)	Genel kullanıcı bilgilerinin saklandığı tablo. Kullanıcıların bilgileri, e-postaları ve şifreleri gibi bilgileri içerir.
OperationClaims (Yetkiler)	Kullanıcıların yetki kısıtlamalarını belirleyen ve yetkilere sahip olmayan kişilerin erişim iznini kontrol eden tablo. Kullanıcıların sistemde yapabileceği işlemleri tanımlar.
UserOperationClaims (Kullanıcı Yetkileri)	Kullanıcılar ile yetkiler arasındaki ilişkiyi tutan tablo. Kullanıcıların hangi yetkilere sahip olduğunu belirtir.
Companies (Şirketler)	Şirket bilgilerinin saklandığı tablo. Şirket adı, adresi, vergi numarası gibi bilgileri içerir.
AccountReconciliations (Hesap Uzlaşmaları)	Hesap uzlaşmalarının kaydedildiği tablo. Şirketler arası finansal uzlaşmaların başlatılmasını ve yönetilmesini sağlar.
AccountReconciliationsDetails (Hesap Uzlaşma Detayları)	Belirli tarih aralıklarında hesap uzlaşma yapmak ve detayları saklamak için kullanılan tablo. Her bir uzlaşmanın ayrıntılı bilgilerini içerir. Bu sayede detay listesine göre isterse şu kısımda bir hata/yanlışlık var gibi bir mesaj gönderebilecek.
BaBsReconciliations (BaBs Uzlaşmaları)	BaBs uzlaşmalarının kaydedildiği tablo. Alım ve satım beyannamelerinin karşılaştırılması ve uzlaşma sağlanır
BaBsReconciliationDetails (BaBs Uzlaşma Detayları)	BaBs uzlaşmalarının detaylarının kaydedildiği tablo. Her bir BaBs uzlaşmasının ayrıntılı bilgilerini içerir.
UserCompanies (Kullanıcı Şirketleri)	Kullanıcılar ile şirketler arasındaki ilişkiyi tutan tablo. Hangi kullanıcının hangi şirketle ilişkili olduğunu belirtir.
CurrencyAccounts (Cari Firmalar)	Şirketlere ait finansal hesap uzlaşmalarının saklandığı tablo. Şirketlerin müşterileri veya tedarikçileriyle olan finansal hesap bilgilerini içerir.
Currencies, (Döviz Cinsleri)	Dolar, Euro, TL gibi döviz cinslerinin tutulduğu tablo. Farklı döviz cinslerini tanımlar ve yönetir.
MailParameters (Mail Parametreleri)	Otomatik mail gönderimi için gereken ayarların saklandığı tablo. SMTP sunucu ayarları, e-posta adresleri ve şifreler gibi bilgileri içerir.
MailTemplates (Mail Şablonları)	Otomatik olarak gönderilecek e-mailler için şablonların saklandığı tablo. E-posta içerik şablonlarını tanımlar ve yönetir.
TermsandConditions (Şartlar ve Koşullar)	Kullanıcıların kabul ettiği şartlar ve koşulların saklandığı tablo. Sistem kullanımı ile ilgili yasal ve düzenleyici koşulları içerir.

6. Teknoloji Ve Araçlar

6.1 NET Core ve C#

.NET Core, yüksek performanslı ve çapraz platform uyumlu bir framework olarak, finansal uzlaşma sitesinin backend geliştirmesi için tercih edilmiştir. C# dili ile yazılan bu proje, güçlü tip denetimi ve geniş kütüphane desteği ile yazılım geliştirme sürecini hızlandırmıştır.

6.2 Entity Framework

Entity Framework 2008 yılından itibaren Microsoft tarafında geliştirilen ORM aracıdır. Veritabanı işlemleri için Entity Framework Core (EF Core) kullanılmıştır. EF Core, ORM (Object-Relational Mapping) aracı olarak veritabanı ile uygulama arasındaki veri işlemlerini yönetir. Bu sayede, veritabanı işlemleri nesne tabanlı programlama ile daha güvenli ve yönetilebilir hale gelmiştir.

6.2.1 ORM Nedir?

ORM veya Object to Relational Mapping temel olarak veritabanında yer alan tablo ve alanları nesne olarak kullanmamıza imkan veren bir yazılım mimarisidir.

Böylece yazılım geliştirici veritabanı ve SQL komutlarına ihtiyaç duymadan yazılım geliştirebilir.

6.2.2 Neden Kullanılır?

.NET veya herhangi bir programlama dili ile veritabanı uygulamaları yaparken ilk olarak veritabanı bağlantısı yapılır. Daha sonra SQL komutları ile veritabanındaki veriler alınır.

Alınan veriler programlama diline uygun veri yapılarında saklanarak işlem yapılır.

Verilerin programlama diline uygun veri yapılarına dönüştürülmesi sırasında beklenmedik hatalar, sorunlar oluşabilir.

Ayrıca karmaşık veritabanı sorguları geliştirmeyi daha da zor hale getirir.

ORM araçlarının temel kullanım ne Entity Framework gibi ORM araçları farklı veri sağlayıcıları (SQL Server, MySQL, SQLite gibi) için aynı komutları kullanarak işlem yapmaya imkan verir. deni bu ve bunun gibi sorunları ortadan kaldırmaktır.

6.3 SQL Server

Proje kapsamında veritabanı yönetimi için SQL Server kullanılmıştır. SQL, verilerin yönetilmesi, silinmesi, aktif edilmesi ve üzerinde çalışmasını sağlayan veri tabanı yönetim sistemidir. Structured Query Language kelimelerinin kısaltılmışı olan SQL'in Türkçe karşılığı yapılandırılmış SQL ile yapabileceğiniz çok fazla işlem bulunuyor. SQL veri tabanı yönetimi sistemi aracılığı ile gerçekleştirebileceğiniz işlemler şu şekilde sıralanabilmektedir:

- SQL ile veri tabanında sorgu yürüterek veri alınabilir, veri tabanına kayıt eklenebilir, veri tabanındaki kayıtlar güncellenebilir veya silinebilir.
- Kullanıcıların verileri hızlıca tanımlamasına izin verir. Sınırsız sayıdaki veri arasından sorgulama ve arama yapılabilir.
- Kimlerin veri tabanına erişebileceği ayarlanabilir, güvenlik önlemleri alınabilir. İş sorgu dili anlamına gelir.

6.4 API Geliştirme ve Entegrasyonu

API geliştirme ve entegrasyon süreci, sistemler arası veri alışverişini ve işlevselliği sağlamak için kritik bir rol oynamaktadır. Bu süreç, API'ların tasarımı, geliştirilmesi, test edilmesi ve diğer sistemlerle entegrasyonunu içerir.

Hedefler ve Amaçlar:

- Farklı sistemler arasında veri paylaşımını ve işbirliğini sağlamak.
- Uygulama içi ve uygulamalar arası işlevselliği genişletmek.
- Güvenli, ölçeklenebilir ve bakım kolaylığı sağlayan API çözümleri sunmak.

Geliştirme Süreci:

- Tasarım: API'ların veri modelleri, uç noktalar (endpoints) ve istek/yanıt yapıları tasarlanır.

- Geliştirme: RESTful veya SOAP tabanlı API'lar geliştirilir. Programlama dilleri ve framework'ler (örneğin, Node.js, Django) kullanılır.
- Test: API'lar, birim ve entegrasyon testleriyle kapsamlı bir şekilde test edilir. Postman ve Swagger gibi araçlar kullanılarak test senaryoları oluşturulur.
- Dokümantasyon: API kullanımı ve entegrasyonu için ayrıntılı dokümantasyon sağlanır. Swagger veya Apiary gibi araçlar kullanılarak API belgeleri hazırlanır.

Entegrasyon:

- Mevcut sistemler ve üçüncü taraf hizmetlerle API entegrasyonları gerçekleştirilir.
- Entegrasyon süreçleri, veri aktarımının güvenliği ve bütünlüğünü sağlamak için dikkatli bir şekilde planlanır ve uygulanır.

6.5 Veri Yönetimi ve Güvenliği

Veri yönetimi ve güvenliği, sistemin güvenilirliğini ve kullanıcıların gizliliğini sağlamak için kritik öneme sahiptir. Bu bölümde veri doğrulama, güvenlik protokolleri ve erişim kontrolü ele alınmaktadır. Ele alınan başlıklar, sisteminizin API geliştirme, veri yönetimi ve güvenliği ile ilgili süreçlerini kapsamlı bir şekilde açıklar ve kullanıcıların bu süreçler hakkında detaylı bilgi edinmesini sağlar.

6.5.1 Veri Doğrulama

Veri doğrulama, sistemde kullanılan verilerin doğruluğunu ve bütünlüğünü sağlamak için yapılan işlemlerden oluşur.

Hedefler: Verilerin tutarlılığını, doğruluğunu ve güvenilirliğini sağlamak.

Yöntemler:

- Giriş Doğrulama: Kullanıcı girişlerinin ve API isteklerinin doğruluğunu kontrol etmek için form validasyonları ve regex kullanımı.
- Veri Temizleme: Veritabanına kaydedilen verilerin temiz ve standart formatta olmasını sağlamak için veri temizleme işlemleri.
- Tutarlılık Kontrolleri: Veritabanındaki ilişkili tablolar arasındaki veri tutarlılığını kontrol etmek ve uyumsuzlukları gidermek.

6.5.2 Güvenlik Protokolleri

Güvenlik protokolleri, veri aktarımının ve depolanmasının güvenli bir şekilde gerçekleştirilmesini sağlamak için kullanılan yöntemler ve teknolojilerdir.

Hedefler: Veri güvenliğini sağlamak, yetkisiz erişimi engellemek ve veri ihlallerini önlemek.

Yöntemler:

- Şifreleme: Veri aktarımı sırasında HTTPS protokolü kullanılarak SSL/TLS ile şifreleme sağlanır. Veritabanında hassas veriler (örneğin, şifreler) şifrelenerek saklanır.
- Kimlik Doğrulama ve Yetkilendirme: JWT (JSON Web Token) veya OAuth kullanarak kullanıcı kimlik doğrulaması ve yetkilendirme işlemleri gerçekleştirilir.
- Güvenlik Duvarları ve İhlal Tespiti: Sistem güvenlik duvarları ve IDS/IPS (Intrusion Detection/Prevention Systems) kullanılarak ağ güvenliği sağlanır.

6.5.3 Erişim Kontrolü ve İzinleri

Erişim kontrolü ve izinler, kullanıcıların ve sistem bileşenlerinin yalnızca yetkilendirildikleri verilere ve işlemlere erişimini sağlamak için uygulanan politikaları kapsar.

Hedefler: Yetkisiz erişimi engellemek ve kullanıcıların yalnızca izin verilen işlemleri gerçekleştirmesini sağlamak.

Yöntemler:

- RBAC (Role-Based Access Control): Kullanıcı rolleri tanımlanır ve her role belirli izinler atanır.
- ACL (Access Control Lists): Belirli kullanıcılar veya gruplar için erişim hakları belirlenir ve yönetilir.
- Kullanıcı İzinleri Yönetimi: Kullanıcıların erişim hakları ve izinleri düzenli olarak gözden geçirilir ve güncellenir.

6.6 RESTful API İlkeleri

REST API'leri, geliştiricilerin API'leri oluştururken talimatları yerine getirebileceği ve yanıt alabileceği bir dizi kuralına sahip API'ler ve yönetim kurulumudur .

Temsili Durum Aktarımı yani REST, Temsili Durum Aktarımı olarak konumlandırılır. RESTful API olarak da adlandırılır.

Rest API, dosyaları veya cihazların nasıl bağlanabileceğini, parçaların nasıl iletişim kurabileceğini ve bunların arasındaki standartların tanımlanmasını sağlar. Ayrıca çevre bölgelerindeki kodun, sunucunun çalışmasını etkilemeden değiştirmeni ve sunucu genelindeki kodun, genelinin çalışmasını etkilemeden değiştirmeni sağlar.

REST API'leri, bir kaynak içinde kayıt oluşturma, okuma, güncelleme ve silme gibi veri tabanı işlevlerini gerçekleştirmek için HTTP istekleri aracılığıyla iletişim kurar. GET, POST, PUT ve DELETE gibi bu dört temel isteği kullanır.

REST API'ler, hemen hemen her programlama dilini destekler. Tek koşulu 6 REST ilkesine uygun olmalarıdır. Bu ilkeler şunlardır: Tek tip arayüz, istemci-sunucu özerkliği, durum bilgisinin olmaması, önbelleğe alınabilirlik, katmanlı sistem mimarisi ve isteğe bağlı kod yapısı.

Finansal uzlaşma sitesinde kullanılan API'lar RESTful mimari prensiplerine göre tasarlanmıştır. RESTful API'lar, HTTP protokolünü kullanarak kaynakların (resources) temsil edilmesini ve manipüle edilmesini sağlar. Bu mimari, API'ların ölçeklenebilir, esnek ve kolay anlaşılır olmasını sağlar.

6.6.1 Rest Sırasında Kullanılan İstekler Nelerdir?

RESTful API'ler genellikle HTTP kullanılarak uygulanır. Bir HTTP yöntemi, sunucuya kaynağa ne yapması gerektiğini söyler. Rest sırasında kullanılan HTTP istekleri şöyledir;

- GET: İstemciler, sunucuda bulunan verileri görüntülemek ve belirli bir düzende listelemek için GET kullanılır.
- POST: İstemciler, verileri sunucuya eklemek için POST isteğini kullanır. Yani kaynak yaratmak için bu isteği kullanır.
- DELETE: Gereksiz verileri ve kaynakları silmeyi sağlar.
- PATCH: İstemciler, sunucudaki verilerin ve kaynakların güncellenmesi için PATCH isteğini kullanır.

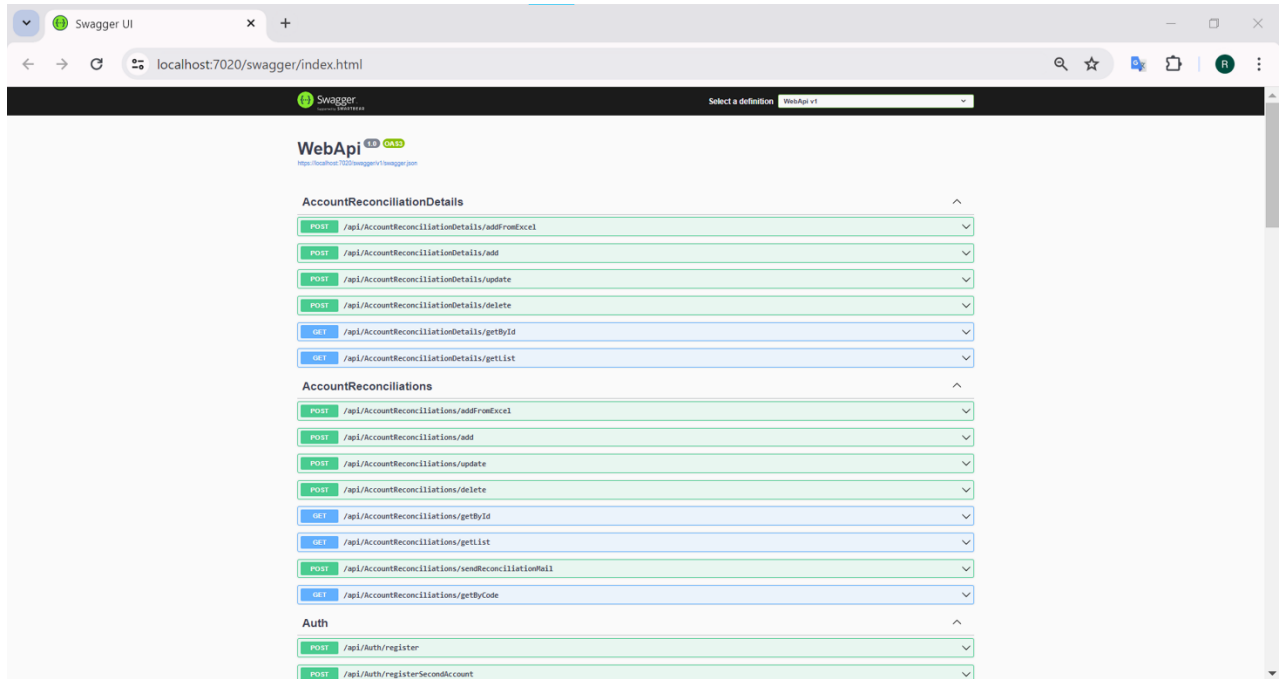
6.7 Swagger ve API Dokümantasyonu

Swagger size Api dokümantasyonu oluşturabileceğiniz bir arayüz sunmaktadır. Sunucunuz aracılığıyla Api yapınızın nasıl çalıştığını geliştiricilerinize veya kullanmak isteyen firmalara açabilirsiniz. Bu sayede geliştiriciler veya firmalar sizin API'leriniz üzerinde farklı uygulamalar geliştirebilirler ya da sizin yazılımınızı kendi yazılımlarına entegre edebilirler.

API'ların dokümantasyonu için Swagger kullanılmıştır. Swagger, API'ların tanımlanmasını ve interaktif dokümantasyon oluşturulmasını sağlar. Bu araç, API geliştiricileri ve kullanıcıları için büyük kolaylık sağlamaktadır.

Swagger'ın bir diğer özelliği de örnek AUTH yani yetkilendirme parametrelerinin tanımlanabilmesidir. Bunun için Swagger üzerinde bir güvenlik mekanizması bulunur ve farklı tokenler burada barındırılabilir.

Swagger tanımlanan istekler üzerinden HTTP örnek istekler oluşturmanızı da sağlar. Yani tanımlanmış bir istek vasıtasıyla sistemdeki yapıları kontrol edebilir ve istek yollayabilirsiniz. Bu özellik sayesinde herhangi başka bir araç olmadan dokümantasyon, test ve geliştirme aşamalarınızı otomatikleştirebilirsiniz.



7.1 CRUD

CRUD işlemleri, bir veritabanı tablosunda oluşturma (Create), okuma (Read), güncelleme (Update) ve silme (Delete) işlemlerini ifade eder. Bu işlemler, veri yönetiminin temelini oluşturur ve finansal uzlaşma sisteminin işlevselliği için kritik öneme sahiptir.

Temelde database de tablelerde yaptığım tüm işlemlerin kısaltması. Bunları biz DataAccess katmanında tanımlarız. Business katmanında ise bu işlemleri çağırarak öncesini ve sonrasını kontrolleri veya işlemleri yazıyoruz.

7.1.1 Create (Oluşturma)

Yeni bir kayıt oluşturma işlemi, kullanıcı veya şirket gibi temel verilerin sisteme eklenmesini sağlar. Örneğin, yeni bir kullanıcı eklemek için aşağıdaki adımlar izlenir:

- Kullanıcıdan alınan veriler doğrulanır.
- Veriler, uygun formatta veritabanına eklenir.
- Yeni kullanıcının başarılı bir şekilde eklendiğine dair geri bildirim sağlanır.

7.1.2 Read (Okuma)

Mevcut verilerin okunması ve görüntülenmesi işlemi, kullanıcıların sisteme eklenmiş verileri incelemesini sağlar. Örneğin, kullanıcı bilgilerini görüntülemek için aşağıdaki adımlar izlenir:

- Veritabanından kullanıcı verileri çekilir.
- Çekilen veriler, uygun formatta kullanıcı arayüzünde görüntülenir.

7.1.3 Update (Güncelleme)

Mevcut bir kaydın güncellenmesi işlemi, kullanıcı veya şirket bilgilerinin değişikliklerini

yansıtır. Örneğin, bir kullanıcının e-posta adresini güncellemek için aşağıdaki adımlar izlenir:

- Kullanıcıdan alınan yeni veriler doğrulanır.
- Veritabanındaki mevcut kayıt güncellenir.
- Güncellenmenin başarılı olduğuna dair geri bildirim sağlanır.

7.1.4 Delete (Silme)

Mevcut bir kaydın silinmesi işlemi, artık gerekli olmayan verilerin sistemden kaldırılmasını sağlar. Örneğin, bir kullanıcı hesabını silmek için aşağıdaki adımlar izlenir:

- Silinecek kullanıcının kimliği doğrulanır.
- Veritabanındaki ilgili kayıt silinir.
- Silme işleminin başarılı olduğuna dair geri bildirim sağlanır.

7.2 Generic Yapı

Temel mantığı bizim her sınıfa ait temel CRUD işlemleri oluşturduk ve her sınıfımızın içerisine girip ayrı ayrı imza oluşturmakla uğraşmak istemediğimiz zaman devreye Generic Yapılar giriyor.

Burada CRUD işlemleri için bir defaya mahsus generic yapı oluşturuyoruz ve sonra yapıyı sınıflar içerisinde çağırıp kullanabiliyoruz.

Generic yapı Core Katmanında oluşuyor. Core katmanına gidiğ DataAccess adında bir folder oluşturuyoruz sonra o folder a bir IEntityRepository adında interfaces ekliyoruz.

Normalde biz `void Add(Company company);` yazardık ama şimdi
Şimdi `void Add(T entity);`

Burada bizim T yi kullanabilmemiz için public yanına başa aşağıdaki kodu ekliyoruz.

```
public interface IEntityRepository<T> where T : class, IEntity, new()
```

Bunun açıklaması diyoruz ki IEntityRepository bir tane T al bu T de class olmalı, IEntity ile implant edilmeli yani IEntity buna eklenmiş olmalı. Aynı zamanda da new lenebilir olsun. Yani ben buraya interfacesler gönderemeyim onlar newlenemez ama classlar newlenebilir.

Artık biz bunu bir Generic yapıda oluşturmaya başladık.

```
List<T> GetList(Expression<Func<T,bool>>filter = null);
```

Burada biz ben bir sorgu yapabilirim bu sorgum aynı zamanda null da olabilir. İki durumda da bunu çalıştır dedik.

Daha sonra gittik Core katmanı içerisine bir DataAccess klasörü oluşturduk. O klasörün içine de : IEntityRepositoryBase sınıfı ekledik. Bu sınıf içerisine Tentity ve Tcontextlerimizi ayarladık.

Artık önceden CRUD işlemleri için oluşturduğumuz IcompanyDal içine tek tek CRUDları yazmak yerine artık sadece public yanına : IEntityRepository<Company> yazıyoruz ve böylelikle tek tek yazmaktan kurtulup tanımladıklarımızı yani imzalarımızı tek bu kodla çağırmış oluyoruz.

Şimdi DataAccess deki Concrete içerisinde ki EntityFramework dosyamıza EfCompanyDal isimli bir sınıf ekliyoruz.

Bütün sınıflara aynı şekilde IEntityRepository eklemesi yapıp DataAcces katmanında Abstract içerisine ekliyoruz.

Biz şimdi bu adımlar ile generic yapımızı anladık ve oluşturduk artık sıra generic yapıyı classlara uygulamak. Bunun içinde generic yapımızın concrete katmanı klasörüne tanımlayacağız. DataAcces katmanımızın Concrete içindeki EntityFramework içerisine class oluşturup uygulayacağız. Bunun içinde önceki adımlar gibi her sınıfı isimlendireceğiz bu sefer I yerine hepsinin başına Ef eki ekliyoruz ve bunları IEntityRepository yerine EfEntityRepositoryBase eklemesi ve ContextDb den veri çekmesi için ekleme gerçekleştiriyoruz ardından soyut interfacemizi ekliyoruz birinin örneğini vericek olursam : (Bu işleme implament deniyor.)

```
public class EfAccountReconciliationDal :  
EfEntityRepositoryBase<AccountReconciliation,ContextDb> , IAccountReconciliationDal
```

Bunu her sınıfa uygulayacağız. Böylece Generic yapımızı tamamlamış olacağız.

7.3 Dependency Injection

Bağımlılık enjeksiyonu demektir. Dependency Injection, bir nesnenin ihtiyaç duyduğu, bir diğer deyimle bağımlı olduğu nesneleri direkt olarak kendi içerisinde oluşturmak yerine dışarıdan enjekte etmeyi mümkün hale getiren bir programlama tekniğidir. Dependency Injection bağımlılıkları tamamen bitirmez, fakat bir birilerine sıkı bir şekilde bağlı olan (tightly coupled) iki nesne arasındaki ilişkinin daha gevşek bir yapıya (loosely coupled) kavuşmasını sağlar. Bu da yazılımcıya pek çok açıdan oldukça büyük avantajlar sağlar.

Dependency Injection tekniğinin sağladığı avantajlardan bazılarını daha iyi bir şekilde anlayabilmek için şu örneği verebiliriz. Elimizde A ve B class'ları var. A class'ında B class'ından bir nesne ürettiğimizi var sayalım. Böylelikle A class'ı B class'ına bağımlı bir hale gelmiş oldu. Yani B nesnesi ile ilgili bir değişiklik yapmamız gerektiğinde mecburen A

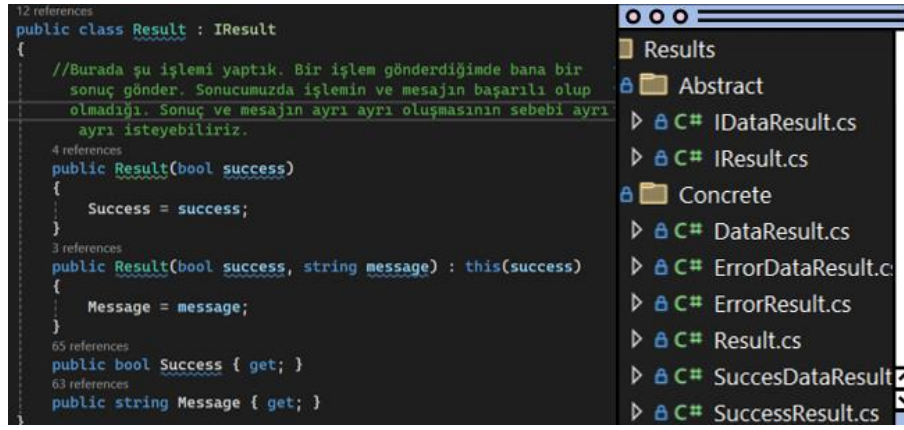
class'ını açmamız ve değişiklikleri uygulamamız gerekecek. Buraya kadar olan kısımda herhangi bir problem görünmüyor. Fakat ya B class'ından türetilen aynı özelliklere sahip bir nesneyi birden fazla class'ta new'lemişsek yani oluşturmuşsak ?

Bunların her biri birbirinden bağımsız objeler olduğu için, tüm class'lara teker teker girip yapmamız gereken değişiklikleri tüm class'lar için uygulamamız gerekecek. Elinizde onlarca class varsa bu işlemler hem yeni hatalara davetiye çıkaracak, hem de daha fazla kod yazmanıza ve doğal olarak daha fazla zaman harcamanıza neden olacaktır. Tüm bunların haricinde kodlarınız reusable yani yeniden kullanılabilir olmadığı için çok fazla kod tekrarı yapmış olacaksınız. Ve son olarak geliştirdiğiniz programı test etmek daha zor ve çetrefilli bir hale gelecek. Görüldüğü üzere her açıdan problemli bir durum çıktı ortaya çıkacak. Farklı class'larda kullanılacak olan objenin yalnızca bir defa oluşturulmasını ve ihtiyaç duyulan class'lara ihtiyaç anında dışarıdan enjekte edilmesini sağlıyor. Böylelikle bu obje üzerinde herhangi bir değişiklik yaptığımız taktirde, tek tek bağlı class'ları gezmemiz gerekmiyor. Yalnızca ve direkt olarak bu obje üzerinde gerekli değişiklikleri yapmamız yeterli oluyor.

Yarın öbür gün mesela Oracle geçmek istersek sadece ICompanyDal → OcCompanyDal diye eşleştirme yapacağız. MySql e çevirirsek de MyCompanyDal diyeceğiz gibi.

7.4 İşlem Sonuç Sistemi

Herhangi birinde işlem yaptığımız zaman hangi işlemi yaptığımızın bir önemi yok. Bu sistem bize desin ki senin bu işlemin başarılı, başarısız al istediğin data al istediğin veri. Bana her yaptığım işlemde istisnasız hangi işlemi yaparsam yapayım bana bu sonucu göndersin. İşte biz buna işlem sonuç sistemi diyoruz. İşlem sonuç sistemizde bir result yapısı oluşturacağız(core katmanına). Bu oluşturduğumuz yapı sonucunda ben hangi veriyi istersem isteyeyim yani hangi sorgu ya da işlemi yaparsam yapayım bu bana sonuç olarak bunlardan (başarılı,başarısız,data,veri) gönderecek. Yani WebApi kısmından Front-End e verdiği veri kısmından bu işlemin başarılı olup olmadığı sonuç olarak bir mesaj eklemek istersem içerisine bir mesaj olup olmayacağı ve en sonda içerisinde bir veri yan gerekirse bir sorgu yapıyorsam select sorgusu gibi onun verisi olacak.

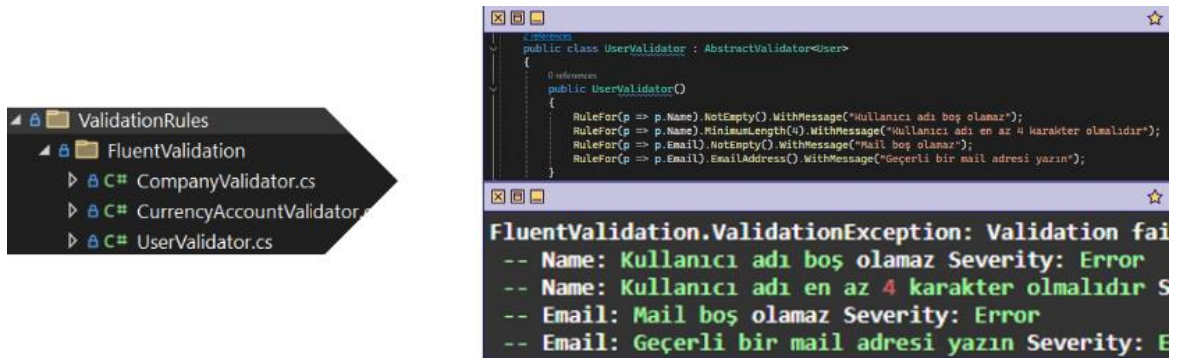


7.5 FluentValidation

Fluent validation; bir nesnenin geçerli olup olmadığını kontrol etmek için kullanabileceğiniz bir doğrulama kütüphanesidir. Bu kütüphane, nesnenin özelliklerini ve bu özelliklerin geçerli olup olmadığını kontrol etmek için sınıflandırılmış bir dizi kural sağlamaktadır.

Fluent Validation, .NET projelerinde kullanıcı girdilerinin kolay ve sade bir şekilde doğrulanmasını sağlayan ve birden çok platformda kullanılabilen çok yönlü ve güçlü bir doğrulama kütüphanesidir. Web uygulamaları, mobil uygulamalar ve masaüstü uygulamalar için fluent validation kullanılabilir.

Fluent validation kullanımının avantajları olarak kompleks şartların gerektiği durumlarda if-else kullanımının karmaşıklığını ortadan kaldırılabilir. Böylelikle yazmış olduğumuz kod daha okunaklı ve anlaşılır hale gelir. Ayrıca kurallarınızı tek bir yerde toplamaya olanak tanır.



7.6 Hashing

Bunlar güvenlik kavramlarıdır. Bir datayı karşı taraf okuyamasın diye yapılan çalışmalardır. Önemli bilgilerimizi saklamak isteriz. Birileri bu bilgilere kolayca erişemesin, erişse bile

kolayca anlamasını isteriz. Ya da birisine bir bilgi gönderirken gönderdiğim bilginin gizli ve doğru bir şekilde karşı tarafa ulaşmasını isteriz.

Hash; bir metinden üretilen başka bir metin veya sayıdır. Ortaya çıkan metin veya sayı sabit uzunlukta ve girdideki içeriğe göre çok geniş çapta değişen yapıdadır. En iyi hash algoritmaları hash kodunu orjinal haline geri dönüştürülemeyecek şekilde üretmek için tasarlanmışlardır. En popüler hashing algoritmaları; MD5, SHA, SHA-2'dir. Sistemler parolaları veritabanlarında saklarlar ve bunları şifreli olarak saklarlar. Hashing işlemi parola saklama sürecine çok uygun bir yapıdadır. Çünkü saklanan hash verisinin geri dönüştürülmesi imkansızdır.

Bu tarz verileri saklarken oluşan hashi tahmin etmesi zor hale de getirmek gerekebilir. Yani brute force yöntemler ile saklanan hash verisini tahmin edebilirler. Bunu önlemek için hash işlemi yapmadan önce veriye salt(tuz) diye adlandırılan farklı bir veri daha eklenir. Bu sayede brute force yöntemi ile tahmin etmeyi daha da zorlaştırmış oluruz.

HashingHelper : Core katmanında Utilities klasörüne Hashing adında bir klasör ekliyoruz. Orayada HashingHelper adında bir class ekliyoruz. Bizim göndereceğimiz şifreyi burada salt ile beraber birleştirip bir şifre oluşturacağız. Sonrada Login işleminde de biz şifremizi tekrardan alıp tekrardan bir hashleme ve salt ile beraber kriptolu şifre haline dönüştüreceğiz ve harf harf kontrol ettireceğiz. Burada ilk öncelikle bir şifreleme oluşturduk. Daha sonra kriptoladık ve harf harf tek tek kontrol ettirdik.

CreatePasswordHash (Parola Karması Oluşturma): Girdi olarak düz metin bir parola alır. Çıktı olarak da karma verir.

VerifyPasswordHash (Parola Karması Doğrulama):Tüm karakterler eşleşirse, parola doğru kabul edilir ve true döndürülür; aksi halde false döndürülür.

```
namespace Core.Utilities.Hashing
{
    3 references
    public class HashingHelper
    {
        2 references
        public static void CreatePasswordHash(string password, out byte[] passwordHash, out byte[] passwordSalt)
        {
            using (var hmac = new System.Security.Cryptography.HMACSHA512()) //burada password oluşturduk.
            {
                passwordSalt = hmac.Key;
                passwordHash = hmac.ComputeHash(Encoding.UTF8.GetBytes(password));
            }

            //password kontrol etme işlemi Burada tekrardan kriptoladık.
            1 reference
            public static bool VerifyPasswordHash(string password, byte[] passwordHash, byte[] passwordSalt)
            {
                using (var hmac = new System.Security.Cryptography.HMACSHA512(passwordSalt))
                {
                    var computeHash = hmac.ComputeHash(Encoding.UTF8.GetBytes(password));
                    for (int i = 0; i < computeHash.Length; i++) //Gönderdiğimiz harfleri tek tek karşılaştıracamız.
                    {
                        if (computeHash[i] != passwordHash[i])
                        {
                            return false;
                        }
                    }
                }

                return true;
            }
        }
    }
}
```


7.7 Caching

Caching, sıkça erişilen verilerin hızlı bir şekilde tekrar elde edilebilmesi için geçici bir depolama alanında saklanması işlemidir. Bu sayede, aynı veri tekrar ihtiyaç duyulduğunda daha hızlı erişilir ve performans iyileştirilir.

CacheAspect sınıfı, belirli bir süre boyunca (örneğin 60 dakika) bir metodun sonucunu önbellekte saklayan bir kesme (aspect) mekanizmasıdır. Bu, aynı metodun tekrar çağrıldığında önbellekten hızlıca cevap dönmesini sağlar.

Bu yapı, tekrarlayan veri erişim işlemlerinin performansını artırmak için kullanılan etkili bir mekanizmadır.

```
namespace Core.Aspects.Caching
{
    22 references
    public class CacheAspect : MethodInterception
    {
        private int _duration;
        private ICacheManager _cacheManager;

        21 references
        public CacheAspect(int duration = 60) //sen 60 dk boyunca sistemde kalmışsın
        {
            _duration = duration; //önbellekte saklama süresini (duration)
            _cacheManager = ServiceTool.ServiceProvider.GetService<ICacheManager>();
        }

        2 references
        public override void Intercept(IInvocation invocation)
        {
            var methodName = string.Format($"{invocation.Method.ReflectedType.FullName}.
                {invocation.Method.Name}");
            var arguments = invocation.Arguments.ToList();
            var key = $"{methodName}({string.Join(", ", arguments.Select(x => x?.ToString() ??
                "<Null>"))})"; //key: önbellek anahtarı
            if (_cacheManager.IsAdd(key)) //IsAdd: Anahtarın önbellekte olup olmadığını kontrol eder.
            {
                invocation.ReturnValue = _cacheManager.Get(key);
                return;
            }
            invocation.Proceed();
            _cacheManager.Add(key, invocation.ReturnValue, _duration); //Add: Anahtarı ve sonucu önbelleğe
                ekler.
        }
    }
}
```

7.8 Interceptor

- Interceptor'lar, bir metod çağrısını saran ve bu çağrının başlangıcında, sonunda veya hata durumunda belirli işlevselliği gerçekleştiren yapı. OnBefore: Metod çağrısından önce çalıştırılır. Örneğin, loglama veya yetkilendirme kontrolü yapılabilir.
- OnAfter: Metod çağrısından sonra çalıştırılır. Temizlik işlemleri gibi görevler burada yapılabilir.
- OnException: Metod çağrısı sırasında bir hata oluşursa çalıştırılır. Hata loglama gibi işlemler burada yapılabilir.
- OnSuccess: Metod başarılı bir şekilde çalıştıktan sonra çalıştırılır. Başarı durumuna özel işlemler burada yapılabilir.
- Intercept: Metodun kesme işlemini gerçekleştirir. invocation.Proceed() metodu, asıl metodun çalıştırılmasını sağlar. Hata yönetimi ve başarı durumlarına göre ilgili metodlar çağrılır.

```
public class MethodInterception : MethodInterceptionBaseAttribute
{
    /*Öncesinde sonrasında, hata sırasında veya başarılıysa çalışma sırasında işlemler eklememiz lazım ki Attribute
    bu işlemler esnasında ne yapacağını bilsin. */
    4 references
    protected virtual void OnBefore(IInvocation invocation) { } //Metod çağrısından önce çalışacak işlemler.
    2 references
    protected virtual void OnAfter(IInvocation invocation) { } //Metod çağrısından sonra çalışacak işlemler.
    1 reference
    protected virtual void OnException(IInvocation invocation, Exception e) { } //Metod çağrısı sırasında bir hata
    oluştuğunda çalışacak işlemler.
    2 references
    protected virtual void OnSuccess(IInvocation invocation) { } //Metod çağrısı başarılı olduğunda çalışacak
    işlemler.

    // invocation = çağrı
    3 references
    public override void Intercept(IInvocation invocation) //Metod çağrısını kesen ve yukarıdaki işlemleri yürüten
    ana metod.
    {
        var isSuccess = true;
        OnBefore(invocation);
        try
        {
            invocation.Proceed();
        }
        catch (Exception e)
        {
            isSuccess = false;
            OnException(invocation, e);
            throw;
        }
        finally
        {
            if (isSuccess)
            {
                OnSuccess(invocation);
            }
        }

        OnAfter(invocation); //tüm işlemlerden sonra invocation dönsün.
    }
}
```

8. TEST VE DEĞERLENDİRME

8.1 Birim Testleri

Birim testleri, yazılımın en küçük parçalarını (fonksiyonlar, metotlar) bağımsız olarak test etmek için gerçekleştirilir. Bu testler, kodun doğru çalıştığını ve beklenen çıktıları ürettiğini doğrulamak amacıyla kullanılır. .NET Core framework'ü üzerinde geliştirilen uygulama için xUnit test framework'ü kullanılmıştır. Birim testleri, kodun her değişikliğinde otomatik olarak çalıştırılarak, hataların erken tespit edilmesini sağlar.

8.2 Entegrasyon Testleri

Entegrasyon testleri, birimlerin veya bileşenlerin birlikte uyum içinde çalışıp çalışmadığını test eder. Bu testler, farklı modüllerin entegrasyon noktalarındaki olası hataları tespit etmeye odaklanır. Finansal uzlaşma sitesi için, API uç noktaları ve veritabanı ile etkileşimler gibi entegrasyon noktaları test edilmiştir. Postman ve Newman gibi araçlar, entegrasyon testlerini otomatikleştirmek ve sonuçları değerlendirmek için kullanılmıştır.

8.3 Performans Testleri

Performans testleri, yazılımın belirli yükler altında nasıl davrandığını değerlendirmek için yapılır. Bu testler, sistemin performansını, ölçeklenebilirliğini ve yanıt sürelerini ölçer. Finansal uzlaşma sitesi için JMeter kullanılarak, çeşitli kullanıcı senaryoları altında sistemin performansı test edilmiştir. Bu testler, sistemin yüksek kullanıcı yüklerinde bile stabil ve hızlı çalıştığını doğrulamaya yardımcı olmuştur.

8.4 Test Sonuçları ve Değerlendirme

Yapılan testler sonucunda elde edilen veriler, sistemin genel performansını ve güvenilirliğini değerlendirmek için kullanılmıştır. Birim testleri, kodun doğruluğunu ve hata oranını minimize ederken, entegrasyon testleri sistem bileşenlerinin uyum içinde çalıştığını göstermiştir. Performans testleri ise sistemin yüksek yükler altında bile kararlı çalışabildiğini kanıtlamıştır.

Test sonuçları, geliştirilen finansal uzlaşma sitesinin kullanıcı taleplerine uygun olarak çalıştığını ve güvenilir bir hizmet sunduğunu ortaya koymuştur. Bu değerlendirmeler ışığında,

projede tespit edilen eksiklikler ve iyileştirme alanları belirlenmiş ve gerekli düzenlemeler yapılmıştır.

Bu bölümde açıklanan test ve değerlendirme süreçleri, finansal uzlaşma sitesinin kalitesini ve güvenilirliğini artırmak için hayati öneme sahiptir.

9. SONUÇ VE ÖNERİLER

Bu bölümde, finansal uzlaşma sitesinin geliştirilmesi sürecinde elde edilen sonuçlar genel olarak değerlendirilecek, projenin katkıları ve sınırlamaları incelenecek ve gelecekte yapılacak çalışmalar için öneriler sunulacaktır.

Finansal uzlaşma sitesi, işletmelerin hesap uzlaşma süreçlerini dijital ortamda kolay ve güvenilir bir şekilde yönetmelerine olanak tanımak amacıyla geliştirilmiştir. Projenin başlangıcından itibaren belirlenen hedeflere büyük ölçüde ulaşılmıştır. Kullanılan teknolojiler ve araçlar sayesinde sistem, yüksek performanslı, güvenli ve kullanıcı dostu bir yapıda oluşturulmuştur. Geliştirilen API'lar ve entegrasyonlar, kullanıcıların ihtiyaçlarına uygun olarak tasarlanmış ve test edilmiştir. Bu sayede, işletmelerin mali süreçlerinde verimlilik artışı sağlanmıştır.

9.1 Projenin Katkıları ve Sınırlamaları

Katkılar:

- Verimlilik Artışı: Finansal uzlaşma sistemi, manuel hesap uzlaşma süreçlerini otomatikleştirerek işletmelere zaman ve maliyet tasarrufu sağlamıştır.
- Güvenlik: Kullanılan güvenlik protokolleri ve veri doğrulama mekanizmaları sayesinde, sistemin güvenilirliği artırılmıştır.
- Kullanıcı Dostu Arayüz: Kolay kullanılabilir ve anlaşılır bir arayüz ile kullanıcı deneyimi iyileştirilmiştir.
- Esneklik ve Ölçeklenebilirlik: .NET Core, EF Core ve Docker gibi teknolojiler kullanılarak esnek ve ölçeklenebilir bir yapı oluşturulmuştur.

Sınırlamaları:

- Başlangıç Maliyetleri: Projenin başlangıç aşamasında kullanılan teknolojiler ve araçlar, belirli bir maliyet getirmiştir.

- Öğrenme Eğrisi: Yeni teknolojiler ve araçlar, geliştirici ekip için başlangıçta bir öğrenme eğrisi oluşturmuştur.
- Kullanıcı Eğitim İhtiyacı: Kullanıcıların yeni sisteme adapte olabilmesi için eğitim ve destek ihtiyacı doğmuştur.

9.2 Gelecek Çalışma için Öneriler

Mobil Uygulama Geliştirme: Kullanıcıların hesap uzlaşma işlemlerini mobil cihazlar üzerinden de yapabilmeleri için bir mobil uygulama geliştirilmesi önerilmektedir.

Yapay Zeka ve Makine Öğrenimi Entegrasyonu: Uzlaşma süreçlerini daha da iyileştirmek ve otomatik hale getirmek için yapay zeka ve makine öğrenimi algoritmalarının entegrasyonu değerlendirilebilir.

Uluslararası Pazara Açılma: Sistemin farklı dillerde ve para birimlerinde çalışabilir hale getirilerek uluslararası pazarlara açılması sağlanabilir.

Güvenlik ve Performans İyileştirmeleri: Sürekli güvenlik güncellemeleri ve performans optimizasyonları yapılarak sistemin güncel ve güvenli kalması sağlanmalıdır.

Kullanıcı Geri Bildirimleri: Kullanıcılardan düzenli olarak geri bildirim alınarak, sistemde gerekli iyileştirmeler ve yeni özellikler eklenmelidir.

Bu öneriler, finansal uzlaşma sitesinin gelecekte daha da geliştirilmesi ve genişletilmesi için bir rehber niteliğinde olup, projenin sürdürülebilirliğini ve başarısını artırmaya yönelik önemli adımları içermektedir.

KAYNAKLAR

<https://medium.com/kodluyoruz/asp-net-core-ile-%C3%A7ok-katmanl%C4%B1-yap%C4%B1-ve-veri-taban%C4%B1-entegrasyonu-fe3444c5271a>

<https://arcayazilim.com/finansal-uzlaşma-sistemi/>

<https://coderspace.io/sozluk/api>

<https://coderspace.io/sozluk/rest-api#:~:text=REST%20API'ler%2C%20hemen%20hemen,ve%20iste%C4%9Fe%20ba%C4%9Fl%C4%B1%20kod%20yap%C4%B1s%C4%B1>

<https://dergipark.org.tr/en/download/article-file/1468404>

<https://dergipark.org.tr/en/download/article-file/948990>

ÖZGEÇMİŞ

Kişisel Bilgiler

Soyadı, adı : FİLİZ, Rümeysa
Uyruğu : T.C.
Doğum tarihi ve yeri : 08.06.2001 Tekirdağ
Medeni hali : Bekar
Telefon : +90 (533) 554 96 55
e-mail : rumeysfilizs@gmail.com

Eğitim

Derece	Eğitim Birimi	Mezuniyet tarihi
Lise	Nene Hatun Kız Anadolu İmamHatip Lisesi	2019
Üniversite	Amasya Üniversitesi Bilgisayar Mühendisliği	2020 -

İş Deneyimi

Yıl	Yer	Görev
2022	Tekirdağ Süleymanpaşa Belediyesi	Stajyer
2023-2024	Wguard	Stajyer (Sistem Mühendisi)

Yabancı Dil

İngilizce

Hobiler

Resim, felsefe, kitap okumak